面向Skills编程–淘宝企业购端对端研发提效实践

行业运营技术 大淘宝技术 2026年6月16日 23:19 浙江 25人

本文提出"面向Skills编程"范式——将领域知识、工作流、约束规则封装为可版本化的Skills单元，让LLM在确定性框架内生成代码，实现从"人写代码"到"人沉淀Skills，AI写代码"的研发范式升级。以企业购客户对接场景为实战验证，通过项目评估、技术方案、代码生产的研发SOP构建，实现商品域端到端交付周期缩短65%，代码一次生成成功率达90%。

01 核心理念：从“配置化编程”到“面向Skills编程”

传统范式的天花板

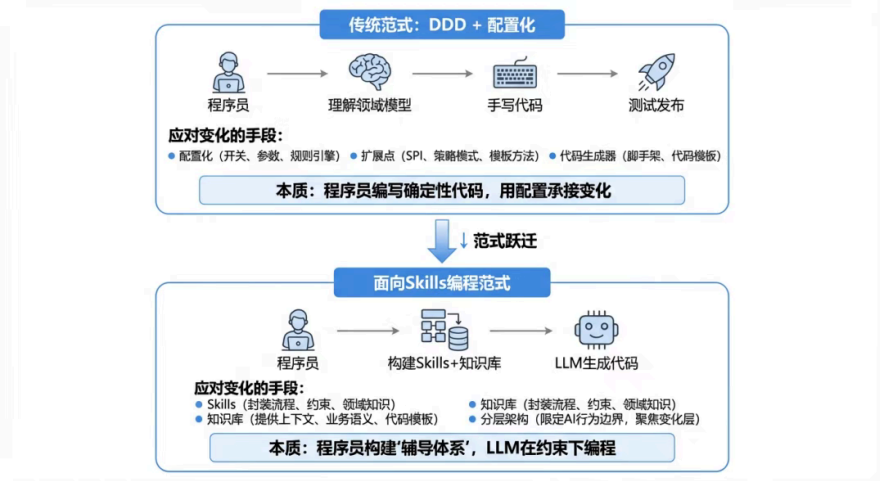
过去我们应对业务变化的经典策略是**DDD分层 + 配置化**：通过领域建模拆分业务能力，通过配置参数驱动行为差异。这套模式在以人为主的传统研发模式下行之有效，但面对高频定制化需求时，由于人力的稀缺，依然可能存在交付瓶颈：



当定制化需求高频出现时（如企业购每个客户接口都不同），配置化的参数空间爆炸，SPI扩展点变成了"每次都要写的代码"，开发者本质上还是在手写适配逻辑—DDD实现了架构解耦，但没有解决**重复编码**的问题。

新范式：面向Skills编程

面向Skills编程的核心思想是：**将"人写代码"转变为"人写Skills，LLM基于Skills写代码"。** 程序员向更高一层抽象——从"实现逻辑"上升为"定义Skills"，基于Skills将个人经验转化为可复用的AI能力单元。



类比理解：如果说传统编程中"接口/抽象类"定义了代码的契约，那么Skills就是定义了**AI行为的契约**——它告诉LLM"做什么、怎么做、不能做什么"，就像接口定义告诉实现类"必须实现什么方法"，让大模型从“知道分子”成为“行动专家”。

范式对比：从DDD+配置化到面向Skills编程

维度	DDD + 配置化	面向Skills编程
应对变化的方式	参数配置 / SPI扩展点	Skill封装不变量，变化的部分作为输入交给LLM
开发者的核心工作	手写业务代码	维护Skills（工作流、知识、约束）+ 审核AI产出
知识载体	文档 + 个人脑中经验	版本化的SKILL.md + references目录
新人上手方式	读文档 + 老人带 + 踩坑积累	读Skills即了解全流程，AI按Skills执行
跨项目复用	复用代码框架，定制部分仍需手写	复用整个Skill体系，定制部分由AI填充
质量保障	CR + 测试	Skill约束 + AI自审 + 交叉验证 + CR

面向Skills编程的通用架构

垂直领域的Skills本身不通用，但**构建Skill的方法论是通用的**——识别重复模式 → 封装不变量为Skills→ 将变化的部分作为输入 → LLM在约束下执行。

- 维度评估：你的场景是否适合引入Skills



- 落地路径：四步构建你的Skill体系

不论你在什么业务域，都可以按以下四步构建自己的Skill体系：



02 实战验证：企业购对接的AI研发演进路径

业务背景

面向企业客户的采购需求，淘宝企业购提供一整套包含淘宝全平台丰富供给以及私有化企业采购服务产品链路的解决方案。在标准模式（SKA模式）下，淘宝企业购已提供完善的基于淘宝开放平台的**TOP标准接口**，支持外部客户自主对接，覆盖标准化的商品、交易和结算服务。

然而在实际业务拓展中，大量**大中型客户**自身已经具备成熟的系统和接口规范：

- **自研内部商城**：如某大型政府机构，内部自建了采购商城系统，拥有自己的接口协议
- **外部SAAS商城**：如使用第三方SAAS采购平台的客户，遵循SAAS平台的接口规范

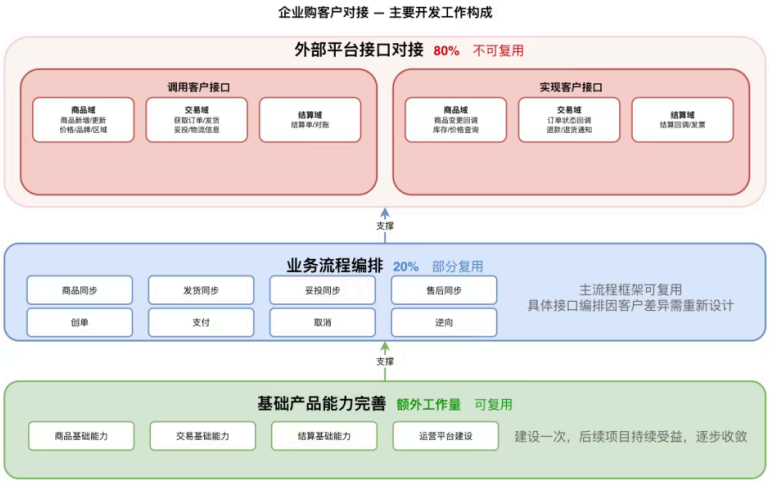
这些客户改造自身系统来适配淘宝企业购标准接口成本较高，因此需要企业购反向适配其系统接口。出于业务发展的需要，企业购需要主动适配客户系统的接口规范，由此衍生出**客户定制对接**这一高频研发场景。相对于客户主动对接TOP标准接口的标准对接模式，我们将这种企业购反向适配客户接口的模式定义为**定制对接模式**。

客户对接的核心工作是：基于客户提供的对接文档，理解其接口规范，开发适配层代码，实现企业购标准服务与客户系统的互通。一次典型的对接项目工作流程如下：



对接工作涉及三大业务域——**商品**（商品信息同步、类目映射）、**交易**（订单创建、物流同步、逆向）、**结算**（对账、结算单、发票管理）。由于每个客户的接口规范各不相同，适配代码几乎无法跨项目复用，每次对接都需要从文档评估到代码开发全流程重新执行。

主要工作构成：



核心痛点

这一业务模式具有四大典型特征：**对外对接高频、需求碎片化、技术方案重复度高、交付周期敏感**。在传统研发模式下，面临以下瓶颈：

维度	痛点数据
交付周期长	• 单项目最长路径 **25个工作日**（从文档评估到最终上线，不含ISV/客户答疑/联调/验收的等待时间）
人力投入大	• 单项投入人多：需商品开发1人、交易开发1人、结算开发1人、测试2-3人，团队还需并行支撑招商、订单贷等常规需求，单人最多仅能支撑1个项目并行 • 无法支持项目并行：开发有一定门槛，涉及商品及交易、结算全流程，**开发经验沉淀在核心开发大脑中**，无法低成本协调其他人员支持
重复工作占比高	80% 工作量集中在外部接口实现（API映射、业务编排），因各平台接口差异大，几乎无法复用；
AI辅助局限	"Vibe Coding"模式依赖开发者个人Prompt技巧，输出质量波动大，难以规模化推广

核心问题：当前以人工为中心、AI辅助的研发模式无法满足业务"快交付、低成本、高灵活"的诉求。

成果速览

经过近半年的系统性探索与实践（2025.8 – 2026.2），团队在**研发范式、研发效率、技术沉淀**三个维度取得了阶段性成果：

- 研发范式升级

从"**依赖个人经验的对话式编程**"迈向"**基于Skills的工程化AI协作**"，实现端对端编程实践探索，以人为主导，逐步转向人通过Skills传递经验给AI，指挥AI编程。

维度	传统模式	AI驱动模式
开发者角色	编码执行者——手写代码、AI辅助补全	辅助AI，指导者和决策者——辅助AI研发，并审核AI输出
代码来源	人工编写为主，AI片段补全为辅	AI生成为主（90%），人工审核为辅
知识管理	散落在个人对话和脑中，不可复用	结构化Skill体系，版本可控、跨项目复用
核心范式	Vibe Coding（依赖个人经验的对话式编程）	Spec Coding（基于规范的工程化AI协作）和Skills coding（可复用的AI能力单元）

- 研发效率提升

商品域端到端交付周期从**23.5人日**缩短至**8人日**，整体提效**65%**，代码一次生成成功率达到**90%**。

指标	优化前	优化后	提效
商品域端到端交付	23.5人日	8人日	**65%**
开发环节	11人日	3人日	**72%**
技术方案	3人日（人工编写）	0人日（AI生成）	**100%**
代码一次生成成功率	不足50%	90%	—
AI代码采纳率	40%	90%	—

- 技术沉淀

构建了覆盖全链路的Skills体系和端到端生码平台，将个人经验转化为可复用的组织能力资产。

沉淀方向	具体产出
全流程Skill体系	18个Skill（通用2 + 商品域15 + 工程1）、10个自动化脚本、10+领域知识文件，覆盖从文档评估→技术方案→代码生成的全链路
端到端生码平台	基于Skill云端化构建的AI研发生码平台（Demo已完成），支持非技术人员通过上传客户文档即可自动生成评估报告、技术方案和代码生产
可复用方法论	8条经过验证的AI研发方法论（工程优先、结构化输入输出优先、清单驱动开发等），其他业务域可借鉴

演进路径：AI研发提效的五阶段

过去半年，团队在巨大的交付压力下，围绕 **架构解耦、AI编程、AI工具应用** 三大方向进行了系统性探索。演进路径与AI技术发展趋势高度一致，每一阶段都是对前一阶段"天花板"的突破：



阶段1：Vibe Coding——以对话驱动需求具象化

> 2025年8月 | 落地项目：某大型ISV

做法：基于弹外架构独立部署的安全性优势（可使用所有开源代码、最新大模型和AI工具），团队开始尝试以自然语言对话驱动代码生成。开发者通过与AI对话描述业务需求，AI直接生成适配代码。

效果：迈出了AI编程的第一步，验证了对话式编码在外部接口适配场景中的可行性。基于某ISV项目完成了整体架构落地，实现企业购标准服务与客户适配逻辑解耦。

瓶颈：

- **产出质量不稳定**：完全依赖个人Prompt技巧，同一需求不同人写出的Prompt质量差异巨大
- **难以复用**：知识散落在临时对话中，新人无法继承前人的经验
- **强依赖个人能力**：AI输出质量与开发者的提示词工程能力强相关

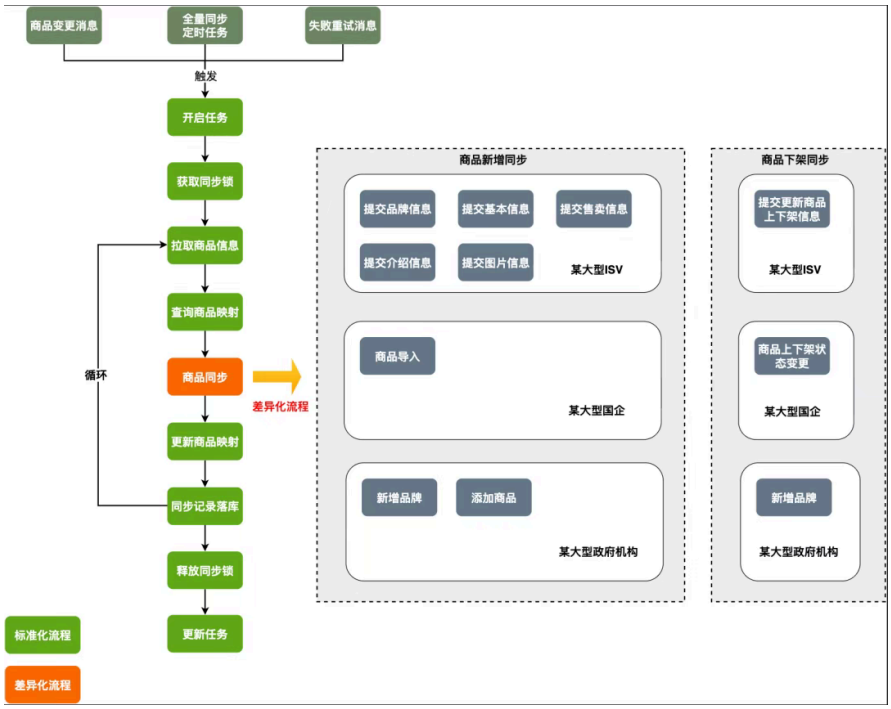
阶段2：Prompt模板——标准化"业务→技术"的语义翻译器

做法：采用**流程模板化 + 子任务Prompt模板**双轨策略，取得阶段性成效：

- **流程模板化**：将共性环节（如"商品同步"）抽象为可复用的对接模板——抽象商品同步流程，分离变与不变的逻辑，对接新客户仅需在已有模板基础上开发变化部分，即可快速生成可用流程，避免从零编排。其核心目的是**让AI做确定性的具体事情，避免发散**——通过预定义流程骨架，将AI的工作从"自由设计一个同步流程"收敛为"在已有模板上填充差异化逻辑"。这本质上是将"参数驱动"的旧范式，升级为"任务视角"的结构化表达——以终为始，围绕业务目标组织流程节点。
- **子任务Prompt模板**：将开发过程拆分为多个子任务（脚手架搭建、接口对象生成、对象映射、接口串联），为每个子任务预置结构化Prompt模板，统一输入格式、约束输出维度、嵌入业务规则。此举显著降低了AI调用的随意性，提升了结果一致性与可维护性，是模板化在认知层的延伸补充。

示例：通过商品域主流程抽象，分离变化的部分和不变的部分，收敛AI的任务

- 流程抽象



效果：

- AI生成代码采纳率 **70%**
- 4类Prompt模板形成可复用资产，新人可快速上手

瓶颈：

Prompt模板虽然在单个子任务上效果显著，但暴露出两个核心问题：

问题一：AI发散不可控，无法提前预览设计思路

Prompt模板只约束了输入和输出格式，但无法约束AI中间的"思考过程"。开发者无法在代码生成前预览AI的设计思路（如接口如何拆分、DTO如何设计、调用顺序如何编排），导致AI生成的代码与预期不符——**发现问题时代码已经生成完毕，只能推翻返工**，反而拉长了迭代周期。

问题二：单点提效，无法承载SOP

Prompt模板无法支撑端到端的标准化操作流程（SOP），核心原因在于：

- **执行不可控**：Prompt模板受限于上下文长度和状态记忆缺失，易出现步骤跳过、顺序错乱、工具误调用等问题，无法保障流程的确定性执行
- **SOP与推理任务的本质冲突**：SOP本质是有约束的确定性流程，而非开放性推理任务——需要将"自由发挥"转化为"受控选择"，但Prompt模板缺乏这种约束能力
- **缺乏能力基座**：当前仍依赖人工保证流程正确性，根源在于缺乏流程节点标准化、工具参数模板化、决策边界显性化的能力基座

阶段3：SDD（规范驱动开发）——构建研发流水线的“数据契约”

> 2025年12月 | 落地项目：SAAS项目

做法：引入SDD方法论，用结构化规格文档驱动AI生成。在SAAS项目中使用OpenSpec工具验证了Spec编程的可行性——从对话式编程到用规范约束AI，通过工程规范、开源知识库、业务约束三层引导，让AI输出能落地的代码。

效果：

模块	集团复用方案	AI + 开源方案	提效
账号系统	30人天	15人天	**50%**
审批流	25人天	8人天	**68%**

- AI生成代码可用率从 **40%提升至80%**

瓶颈：

SDD方法论验证了Spec驱动的可行性，但在实际落地过程中暴露出规模化推广的瓶颈：

- **流程执行成本高**：SDD要求严格遵循"提案（Proposal）→ 实施（Apply）→ 归档（Archive）"三个环节顺序执行，且过程中涉及多轮需求澄清和方案迭代，单个Spec从编写到定稿往往需要3–5轮人工交互，流程本身的投入不可忽视

- **强依赖个人经验，无法规模化**：Spec的编写质量高度依赖编写者对领域知识的掌握程度——如SPU/SKU模型理解、推拉模式选择、字段映射方向等关键决策，仍然散落在个人脑中，缺乏标准化的知识载体。新人面对同一份客户文档，写出的Spec质量与老手差距悬殊，无法通过简单培训弥合
- **领域知识未固化**：SPU/SKU混淆、ID字段混淆等问题跨项目反复出现，说明领域经验没有沉淀为可复用的约束规则，每次仍需人工记忆和校验

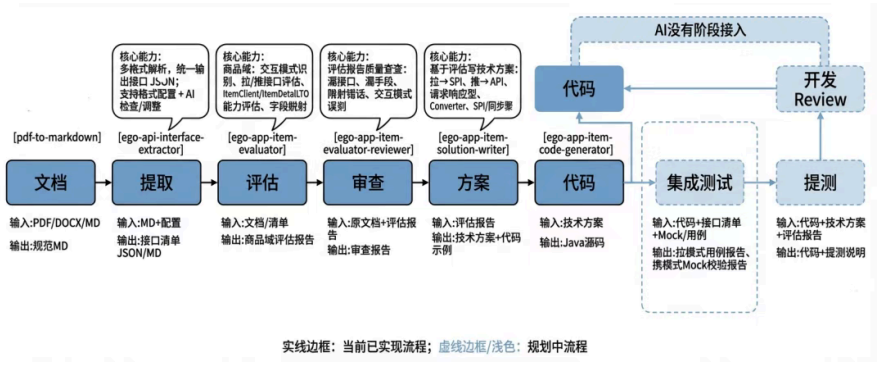
阶段4：Skill沉淀——将经验固化为可复用的AI能力单元

> 2026年1–2月｜落地项目：某大型ISV

做法：企业购客户对接场景天然适合Skill体系——每一次客户对接都是一个全新的项目，但执行的流程高度重复（文档评估→技术方案→代码开发），变化的只是客户接口规范、字段映射关系和业务流程编排。这与Skills倡导的**可复用技能包**理念高度契合：将不变的流程、规则、领域知识封装为Skill，每次对接只需输入客户文档，Skill即可驱动AI按标准化流程产出结果。

基于这一判断，团队采用Anthropic的Agent Skills标准（SKILL.md + references/ + scripts/）将领域经验封装为Skill，实现"**经验即代码**"——工作流写在SKILL.md里，领域知识放references目录，通过版本控制管理，换人、换模型、换平台都能复用。

最终构建了一条从**客户接口文档评估到代码生产**的AI全链路流水线，覆盖"文档评估 → 技术方案 → 编码"的完整链路，用xx项目（15个接口）跑通了商品域整个流程。



效果：

指标	目标	实际
端到端整体提效	38%（23.5d→14.5d）	**65%（23.5d→8d）**
开发环节提效	54%（11d→5d）	**72%（11d→3d）**
一次代码生成成功率	80%	**90%**

瓶颈：

Skill体系在技术研发侧取得了显著成效，但其使用门槛决定了受众仍局限于技术同学：

- **面向技术同学，产品和业务上手门槛高**：Skill的安装、配置、调用均依赖Cursor等本地IDE环境，需要理解SKILL.md工作流、references目录结构、脚本执行等技术概念。产品经理和业务运营无法直接使用，仍需研发作为中间人代为执行，AI提效的红利无法辐射到更广泛的角色
- **依赖本地IDE环境，无法规模化推广**：Skill运行在开发者个人的本地IDE环境中，换人、换机器需要重新配置环境，团队协同和知识同步成本高
- **能力无法产品化输出**：技术侧已验证的全链路能力（文档评估→方案生成→代码生产）缺乏产品化载体，无法作为标准化工具赋能给产品和业务团队自主对接

阶段5：云端集成——打造端到端AI研发产品

> 2026年2月 | 探索中

做法：技术先行，利用OneDay搭建前端交互界面，结合Aone沙箱提供的代码编译与执行环境，自行搭建了端到端的生码平台原型。核心思路是将已验证的Skill能力从本地IDE迁移到云端，让非技术人员通过Web界面即可触发全链路流水线——上传客户对接文档后，平台自动完成文档解析、评估报告生成、技术方案输出、适配代码生产的完整流程。

目标：快速跑通端到端链路，验证Skill云端化的可行性，降低使用门槛，让产品、业务等非技术角色也能直接使用AI研发能力。

当前进展：

- 基于OneDay + Aone沙箱的生码平台已搭建完成，完整链路已跑通
- 商品域全流程（文档评估→技术方案→代码生成）已在平台上端到端验证通过
- 验证了Skill从本地IDE到云端平台迁移的可行性，核心能力可复用

04 方法论沉淀：我们是怎么做到的

从三个核心方向展开，重点分享**可借鉴的经验**：

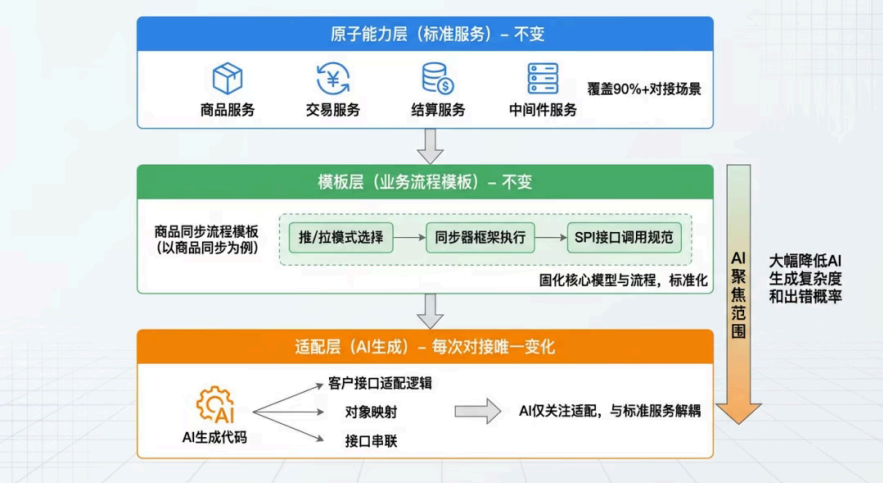
架构先行：分层架构设计

为什么架构优化是AI编程的前提？

代码分层越清晰，AI生成质量越高。架构优化不是为了优化而优化，而是为了让AI能够更好地理解和生成代码。分层架构设计是AI编程的基础设施，它通过清晰的结构和明确的边界，让AI能够像人类开发者一样理解代码的组织逻辑，从而生成更高质量、更符合规范的代码。

- 分层架构设计

通过代码分层，将系统拆分为**原子能力层、模板层、适配层**三层架构。这种分层不仅是传统工程意义上的解耦，更是为AI编程量身设计的——通过流程模板化限定AI行为边界，避免发散，让AI聚焦于专一任务（仅实现适配层逻辑），从而显著提升AI输出的准确性和一致性。



分层	核心价值	具体说明	收益
原子能力层（不变）	封装企业购基础服务能力，提供标准化调用接口	商品查询/同步、交易下单/状态流转、结算对账、缓存/数据库等中间件服务，覆盖90%+对接场景	**零重复开发**：AI和人工均无需重复实现基础能力，直接调用即可
模板层（不变）	固化企业购核心模型与流程，标准化流程	抽象主流程模板（商品全量同步/增量同步流程图、推模式/拉模式、同步器框架、SPI接口规范）	• **业务一致性**：新客户适配均遵循统一标准； • **AI生成基准**：降低逻辑发散风险
适配层（变）	灵活应对客户差异化需求，差异化流程	AI生成差异化代码（对象生成、对象映射、接口串联）；人工补充非标逻辑（如特殊鉴权等）	**聚焦定制需求**：人工开发量减少，AI仅需在固定框架内填充适配逻辑

原子能力层和模板层是稳定不变的，AI生成的代码仅聚焦于适配层——**这将AI的工作范围从"理解整个系统"收敛为"在固定框架内填充适配逻辑"**，大幅降低了AI生成的复杂度和出错概率。

实际效果：

- 适配层代码量减少60%：客户差异化开发聚焦协议转换等核心逻辑
- 多客户并行开发零冲突：支持多项目并行开发与独立部署
- 原子服务复用：覆盖90%+对接场景，AI无需重复生成基础能力代码

垂直领域Skill的构建与调优

- 为什么做垂直领域Skills

回到Skills的本质，构建领域专家，而不是通用方案；围绕企业购做垂直域深耕，真正实现业务提效。

我们所构建的Skills专注于企业购对接这一专业领域的具体需求，与企业购业务强相关——所需的领域知识是SPU/SKU模型、推拉模式、字段映射规则等专业知识，而非通用能力（如PDF解析、代码格式化等通用Skill），选择垂直深耕而非通用方案，基于两个核心判断：

- **垂直域深耕ROI更高**：基于Spec范式、Skills、SubAgent技术体系，围绕企业购项目对接集中人力进行实践和应用，最大程度提升研发效率（质量可控、可复用、可迭代），并通过产品化实现技术赋能业务
- **通用方案短期不可行**：Spec范式的流程验证和可行性验证已经完成，但网站技术域涉及广，做通用的研发方案成本高、短期成功率低，且集团已有较多的专门团队在投入

我们期望的是：**在构建专业垂直领域Skill的过程中沉淀调优经验和方法论，使之可复用于其他领域快速构建Skill**——垂直领域的Skill本身不通用，但构建Skills的方法论是通用的。

- Skill构建思路

- **Spec驱动原则**：通过提升项目评估、技术方案等前置链路准确性，实现生码准确率提升—前序环节的质量决定了后续环节的天花板
- **工程师思维**：先完成链路搭建，再基于实际项目拆解到不同节点进行问题分析，通过工程+AI结合，以解决问题为第一目标
- **借鉴和学习**：参考Anthropic官方最佳实践（三层架构）、优秀开源项目经验（everything-claude-code、superpowers）等进行实践验证

- 实际案例

成功率从早期不足50%提升至当前90%，核心在于解决了**接口提取幻觉、复杂逻辑输出不稳定、长上下文导致信息丢失**三大类问题。调优过程按链路节点逐段攻破：

阶段	主要瓶颈	关键解法	成功率
评估报告	AI幻觉：提取漏接口、领域知识缺失	工程化提取定制（确定性）+ 领域知识注入（知识工程）+ AI审核（不确定性）	50% -> 95%
技术方案	AI输出不稳定：长文档输出崩塌、问题反复、出错概率高	领域架构拆分 + 分批写入 + 清单驱动 + 子Skill拆分（可复用单元）+ 问题沉淀闭环（知识迭代）	70% -> 95%
代码生产	上下文过长：导致领域知识丢失，准确率下降	独立工程初始化Skill（可复用单元）+ 代码模版（知识注入）+ subAgent（架构拆分）	80% -> 90%

评估报告阶段

评估报告是整条链路的输入源头，类似于PRD的理解，这个阶段要解决两个问题：**接口提取不准**和**领域知识缺失**。

问题1：接口提取——AI幻觉导致遗漏

早期完全依赖模型提取接口，模型幻觉导致接口遗漏或误提，错误沿流水线逐级放大——提取漏一个接口，评估就少评一个，方案就少写一个，代码就少生成一个。

典型案例：客户项目A4.5节标题为"获取所有图片信息"，模型未识别为接口定义，直接跳过。

解决方案：脚本提取为主，AI辅助校验

核心思路是把"提取"这个对准确性要求极高的环节从模型能力中剥离出来，改用确定性的脚本解析，AI只负责前后两端（理解格式 + 检查补漏）：



问题2：领域知识缺失——映射关系混乱

AI缺乏商品域业务知识，导致字段映射方向搞反、ID混淆、参数丢失等问题。

解决方案：领域知识内嵌——把人脑中的业务经验转化为Skill的references和约束规则，让AI从"通用地写代码"变成"带着领域知识写方案"。

问题	具体表现	解决方案
SPU/SKU混淆	外部商品指SKU，企业购为SPU挂多SKU，映射方向搞反	references注入字段映射快查表，SKILL步骤写死映射方向
ID字段混淆	外部skuId等同于企业购egoSkuId	快查表明确区分，约束"映射方向：企业购→外部"
字段映射乱序	映射表顺序与原接口出入参不一致	SKILL约束：必须严格按原接口参数顺序排列

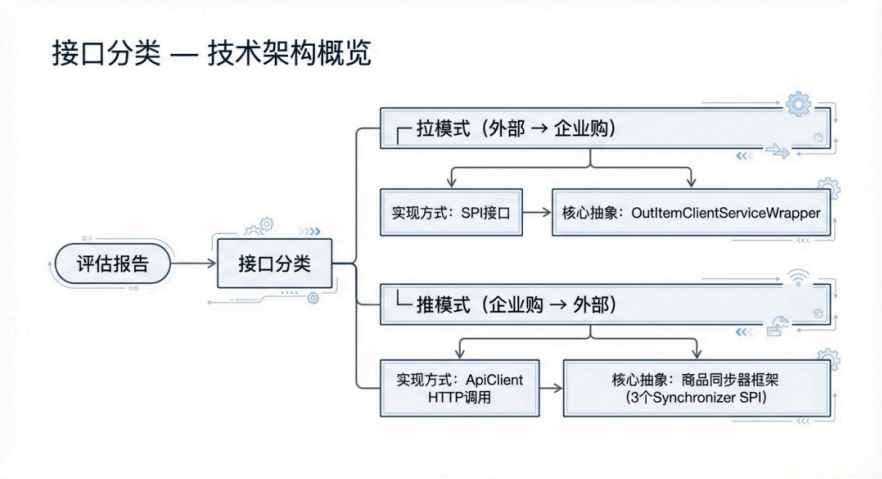
技术方案阶段

评估报告准确后，技术方案阶段暴露出两个新问题：**推拉模式混杂导致API不对齐**和**长文档输出崩塌**。

问题1：推拉模式混杂——生成代码与实际系统API不对齐

早期推模式和拉模式混在一个Skill里生成方案，导致上下文膨胀、AI凭空编造接口签名和DTO结构。

解决方案：领域架构拆分，按调用方向拆分链路 + 系统API抽象注入



维度	拆分前	拆分后
Skill数量	1个混合Skill	推/拉各1个编排Skill + 各4个子Skill
上下文	推拉逻辑交叉，膨胀严重	每条链路独立，上下文干净
API对齐	AI凭空编造接口签名	对齐Wrapper/Synchronizer真实定义
可迭代性	改一个影响两条链路	推拉独立迭代，互不影响

问题2：长文档输出崩塌

某ISV15个接口，AI仅完整实现前4个，后续用"类似"带过。

解决方案：将方案生成Skill拆为4个子Skill，每个接口在独立上下文中处理，本质是**用架构手段解决模型能力边界问题**——不指望模型在超长上下文中保持一致性，而是把问题拆到模型能稳定处理的粒度。

```
pull-generator（编排）
    │
    ├── pull-indexer                脚本解析评估报告 → JSON索引（含子章节行号）
    ├── pull-model-designer         按子章节精确读取参数 → 设计通用基类
    ├── pull-interface-generator × N    单个接口独立处理（每个~8k tokens）
    └── 汇总组装                    拼装N个接口文件 → 最终文档
```

维度	拆分前	拆分后
单次上下文	~50k tokens（8个接口在一个窗口）	~8k tokens/接口（只读该接口的评估章节）
失败影响	整个方案重来	只重做1个接口
并行潜力	不可并行	N个接口可全并行（待SubAgent化）
后段质量	明显下降（注意力不足）	每个接口独立干净，质量一致

问题闭环：同类问题跨项目、跨模型反复出现（如XX系统修了SPU/SKU混淆，换B系统又犯），通过ADJUSTMENT_PLAN机制（发现→定位Skill→补约束→验证→交叉验证）将11类高频问题全部沉淀为Skill约束，不再复现。

代码生产阶段

评估报告和技术方案准确后，代码生产阶段的核心瓶颈是**代码规范和编译成功率**。

问题：AI生成的代码不可用

典型案例：

- 项目骨架未初始化（pom.xml占位符未替换、AKSK未配置），代码放进去编译不过
- AI用`ItemClient`而不是`ItemClientWrapper`（后者封装了ID映射和参数校验），运行时报空指针
- 生成顺序不对：先生成SPI实现类，此时请求/响应基类还不存在，编译报错

解决方案：

- **工程初始化Skill前置**：将项目骨架搭建（占位符替换、AKSK生成、Maven编译验证）拆为独立的`ego-project-initialization` Skill，确保代码生成在一个可编译的工程上进行
- **代码模板驱动生成**：将`ItemClientWrapper`使用方式、工具类API、注解规范、Request/Response/Converter/SPI四类代码模板注入Skill的`references/code-templates/`，AI按模板填充而非凭空编写
- **按依赖顺序生成**：拉模式按"通用基类→请求类→响应类→转换器→SPI实现"的依赖顺序逐接口生成，避免前置类不存在导致编译失败

- 经验总结

Skills构建原则

基础认知：

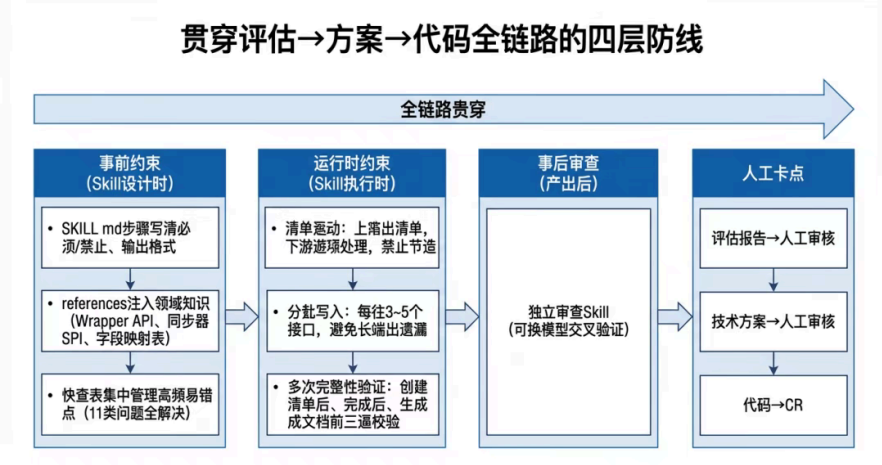
- **Skills 是 AI 研发的最小可复用单元：**类比软件工程中"函数/类"封装逻辑，Skill 封装的是工作流 + 领域知识 + 约束规则—做什么（SKILL.md）、用什么知识做（references）、不能怎么做（约束与禁止项），新客户对接不是从零开始，而是换一份输入文档跑同一条流水线；
- **质量瓶颈不在模型，在知识工程：** 50% → 90% 的提升全部来自知识注入和约束迭代，不是换更强的模型。领域知识（映射规则、API 签名、模式判定）不会从训练数据中涌现，必须显式注入；
- **确定性工程 + 不确定性 AI = 可控的研发流水线**：高精度环节用脚本（接口提取），模型不稳定的用架构拆分绕过（推拉分离、子 Skill），反复出错的沉淀为约束—三者配合把"不可控的对话"变成"可复现的流水线"；

实践经验：

原则	核心做法	解决什么问题
工程优先	能用工程解决的优先用工程，AI进行辅助审查	降低AI幻觉
结构化输入输出优先	链路间传递优先使用代码、JSON、Mermaid等结构化格式	自然语言传递导致下游幻觉
清单驱动开发	上游出结构化清单，下游逐项处理，禁止节选	AI偷懒跳过、生成不全
分批写入策略	长产出分批生成（每批3~5项）	单次输出过长导致后段遗漏
渐进式上下文	SKILL.md定义流程 + references按需加载	上下文爆炸、关键约束被稀释
独立审查机制	生成Skill和审查Skill分离，可换模型交叉验证	自己审自己、单模型系统性偏差
问题沉淀闭环	发现→归类→补约束→验证→交叉验证	同类问题反复出现
进度持久化	所有中间产出写文件，文件即检查点	中断后进度丢失、无法断点续做

质量控制

通过「事前约束→运行时约束→事后审查→人工卡点」四层防线，贯穿评估→方案→代码全链路：



实际效果（xx项目）：审查Skill对评估报告审查结果为接口覆盖15/15、字段遗漏率0%、映射方向正确。早期（xx项目）的11类高频问题已全部通过约束沉淀解决。

知识库建设

- 为什么需要知识库

建设专有知识库，能让AI懂得业务现有的技术背景、领域知识、架构、流程、代码结构等知识，让AI研发更可靠、更"聪明"。在Skill体系中，领域知识通过references目录内嵌到每个Skill中，但随着Skill数量增长，出现了知识分散、更新不同步、跨Skill复用困难等问题，需要构建统一的知识库体系来支撑。

- 知识库全景

知识库采用**三种存储载体**协同的架构，覆盖从代码到文档的全域知识管理：

存储载体	定位与特点	用途
Git知识仓库	代码版本控制平台，面向文档管理，支持多人协作，需手动编写维护	共建的AI研发经验知识：Rules、Skills、Agent配置等
Code Wiki	AI自动生成的代码仓库知识库，与Git仓库强关联，随代码变更自动更新	代码强相关知识：API文档、系统架构、核心模块说明等
KBase	统一知识管理和检索平台，支持关联多种知识源，提供语义搜索	文档数据源知识：集团公开数据资产（如中间件指南）、业务知识、问题案例等

- 知识的生产与使用

生产知识：

- **多模态支持**：钉钉文档、代码、PDF等多种格式
- **多数据源**：Git知识仓库、Code Wiki、KBase等
- **多种生成方式**：人工维护（研发过程中沉淀的Rules、可复用的原子Skills）+ 自动生成&增量更新（如KBase支持的钉钉知识库同步、需求完成后Hooks自动更新）

使用知识：

- AI研发流程自动加载知识，通用Skills隐藏底层检索细节
- Git知识仓库：自建索引，支持渐进式拉取
- KBase：RAG召回（文本块/原文），自然语言提问可召回代码片段+原文档链接

- 标准化知识管理

参考Anthropic官方Skill的渐进式加载策略（SKILL.md触发时加载 + REFERENCE.md按需加载），构建了**三级索引结构**，实现知识的按需发现和加载：

```
ai_coding/
├── GUIDE.md                # 一级索引：使用指导，记录目录结构及索引文件位置
├── rules/                  # 编码规范
│   ├── index.md            # 二级索引：记录所有规则的简单描述
│   ├── alibaba/            # 阿里巴巴编码规范
│   │   └── java编码规则规范.md  # 三级：知识本身，含元数据+使用示例
│   └── growth/             # Growth 业务规范
├── docs/                   # 文档
├── skills/                 # AI 技能配置
├── agents/                 # AI Agent 配置
├── commands/               # 命令配置
└── package/                # 包配置（批量操作）
```

每个知识文件通过标准化元数据（name、description、tags等）实现自描述，索引文件自动生成维护，支持按场景检索和按标签检索。

- 知识分发工具

构建了CLI工具`kn-fetcher`，支持知识的拉取、搜索和批量分发：

```
# 拉取编码规范
kn-fetcher pull --platform aone-copilot --rules java-coding-standards
# 搜索知识
kn-fetcher search --rules java
# 批量初始化（一键拉取项目所需的所有知识）
kn-fetcher pull --platform aone-copilot --package growth-batch-pull
```

通过`init`命令一键完成通用知识的初始化下载，降低新人和新项目的知识配置成本。

- 当前进展

- Code Wiki已完成试跑，生成的数据模型、开发指南、核心模块、API参考符合预期，可通过MCP召回知识
- KBase已完成试跑，自然语言提问可召回代码片段+原文档链接

- 三级索引结构和元数据规范已定义完成
- `kn-fetcher` CLI工具6个基础命令（pull / list / search等）已完成，待对接Skill体系

05 规划与展望：迈向"端到端智能交付"的研发未来

当前评估→方案→编码链路已验证，但沙箱测试（TDD）、SubAgent并行化等能力尚未完全完成，仍有较大的提升空间，下一阶段将继续推进端对端研发闭环，进一步实现交付周期缩短；

优先级	方向	核心目标	当前状态
P0	SubAgent独立上下文	解决上下文爆炸，支撑并行	方案已设计完成，拉模式已拆分验证
P0	AI沙箱测试（TDD）	代码质量从"人工CR"到"自动验证"	沙箱基础链路已打通，待构建测试能力
P1	知识库统一	一处更新多Skill生效	6个基础命令已完成，待对接Skill体系
P1	批量评测体系	量化Skill质量，形成回归基线	待启动
P1	多模型交叉验证	利用模型差异性提高检测覆盖率	审查Skill已支持换模型，待系统化
P1	多Agent并行编排	拉/推并行，端到端耗时缩短	待SubAgent基础完成后启动

06 结语

从Vibe Coding到Skills coding，从50%的代码生成成功率到90%，从23.5人日的交付周期到8人日—这不仅是工具链的升级，更是**研发范式的重构**。

传统思维中，文档是代码的注释；而在SDD思维中，**Spec是源代码**—开发者维护规范，代码由AI生成。开发者的角色从编码执行者转变为**审核者、架构师**。

在Skills编程体系里，Skills是人类最佳实践的能力封装。开发者的角色从AI辅助研发，变成辅导AI进行研发，人类彻底成为指挥AI进行工作的人，人类研发将更多精力投入到架构设计、代码审核、规范设计等更高价值的创造性工作中。

展望未来，当每个领域的最佳实践都能被沉淀为Skills时，意味着个人经验的产品化、标准化和资产化，AI 会真正从“知道分子”成为“行动专家”；AI不是替代者，而是为我们工作的数字专家，帮助我们从重复劳动中解放，聚焦更高价值的创造。

团队介绍

本文作者官亭，来自淘天集团–行业运营技术团队。在企业购业务中，面向企业客户的采购需求，团队正深入构建AI Agent产品，围绕AI运营、AI研发、AI产品三大方向持续突破，以AI驱动企业采购在项目交付、产品体验、经营效率上的全面升级，助力企业采购实现规模化与高质量增长。团队当前急需前端、后端、QA等方向的伙伴，如果你愿意一起来探索，欢迎联系zezhou.jzz@taobao.com。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 7420

留言 1

写留言

Sheva 北京 6月17日 2
最核心的问题是其实是需求澄清